DEPARTMENT OF COMPUTER SCIENCE & SOFTWARE ENGINEERING

BACHELOR OF SCIENCE IN SOFTWARE ENGINEERING (BS SE)

Academic Project Report

COURSE PROJECT SUBMISSION

Snake Game Made in Windows Forms Application

A Desktop 2D Game Implementing Object-Oriented Software Architecture in C# and .NET

Student Name:	Muhammad Ahsan Nasir
Registration No:	FA25-BSE-028
Course Title:	Object Oriented Programming
Section:	Section B
Academic Session:	2025 - 2029
Framework / Tech:	C# | .NET 10.0 | Windows Forms (WinForms)

Submission Date: October 2026 • Developed in Visual Studio 2026 / 2022

1. Brief Introduction to the Game

The **Snake Game** is an interactive desktop game built as a modern Windows Forms application in C# targeting .NET 10.0. The player controls a snake advancing across a 2-dimensional grid. The core objective is to steer the snake to consume food items, thereby earning score points and growing in length, while avoiding boundary walls, self-intersections, and stationary obstacles.

The primary goal of this project is to apply foundational **Object-Oriented Programming (OOP)** principlesâ€”encapsulation, inheritance, polymorphism, and abstractionâ€”within a graphical event-driven architecture. Game rules, state updates, graphics rendering, audio playback, and file persistence are cleanly decoupled into distinct classes.

2. List of Implemented Features

Grid Movement & Double-Buffered Canvas

Smooth discrete movement across a 25×30 grid (20px cell size). The custom `GameCanvas` panel uses double buffering to prevent screen flickering during rapid timer ticks.

Dual Food System & Scoring

Two food types: Regular Food (red circle, 10 points) and Bonus Food (orange circle, 20 points, 20% random spawn probability) to introduce strategic scoring opportunities.

Three Difficulty Presets

Selectable difficulty dynamically alters tick interval and obstacle density: Easy (180ms, 0 obstacles), Medium (110ms, 4 obstacles), and Hard (70ms, 8 obstacles).

Comprehensive Collision Detection

Instantaneous detection of boundary wall impacts, body self-collisions, and obstacle crashes. Includes a reversal safeguard preventing accidental 180-degree self-bites.

Persistent High Score Tracking

Reads the highest record on startup from `highscore.txt`, compares real-time player score against the high score, and persists new records immediately upon achievement.

Windows Multimedia Audio Engine

Continuous background music (`game_music.mp3`) utilizing the native Windows MCI API (`winmm.dll`) with synchronized play, pause, resume, and stop controls.

Flexible Dual-Input Controls

Simultaneous support for **Arrow Keys** and **W, A, S, D**. Direct command-key processing ensures keystrokes are intercepted before button focus changes.

Intuitive Game State Management

Comprehensive UI showing real-time Score, High Score, and Status (Ready, Playing, Paused, Game Over), with Start, Restart, Pause/Resume, and Exit controls.

3. Screenshots of the Running Application

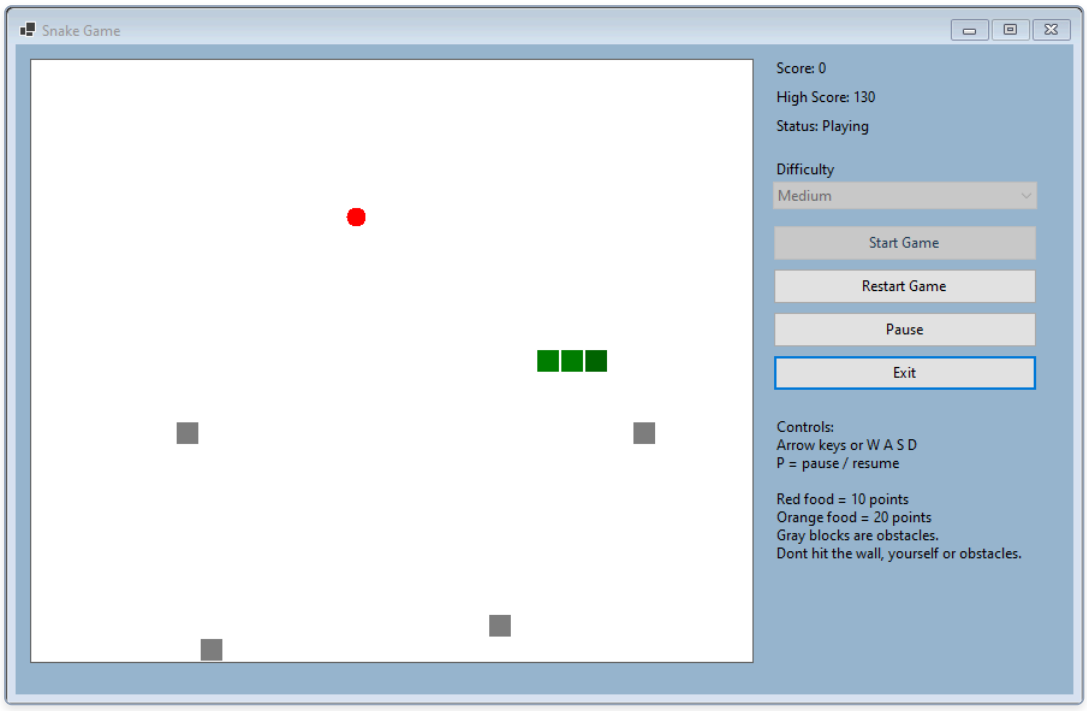


Figure 3.1: Active Gameplay on Medium Difficulty (Snake navigating toward food, obstacles rendered, real-time score tracking)

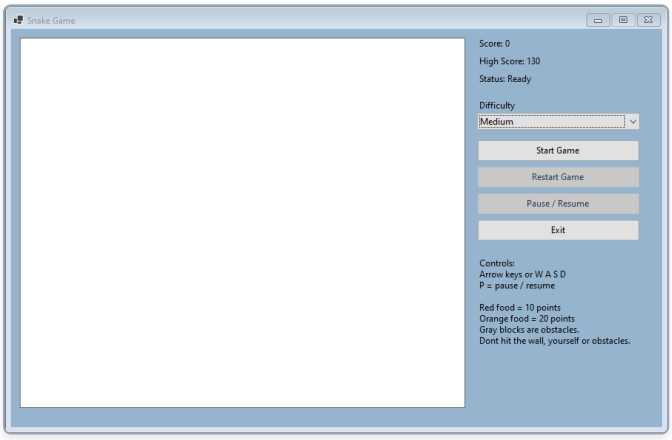


Figure 3.2: Initial Ready State upon Launch (Difficulty selector, Start button, High Score display, Instructions)

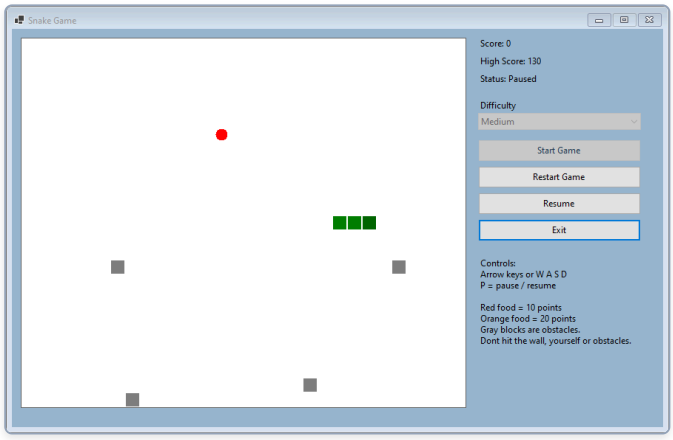
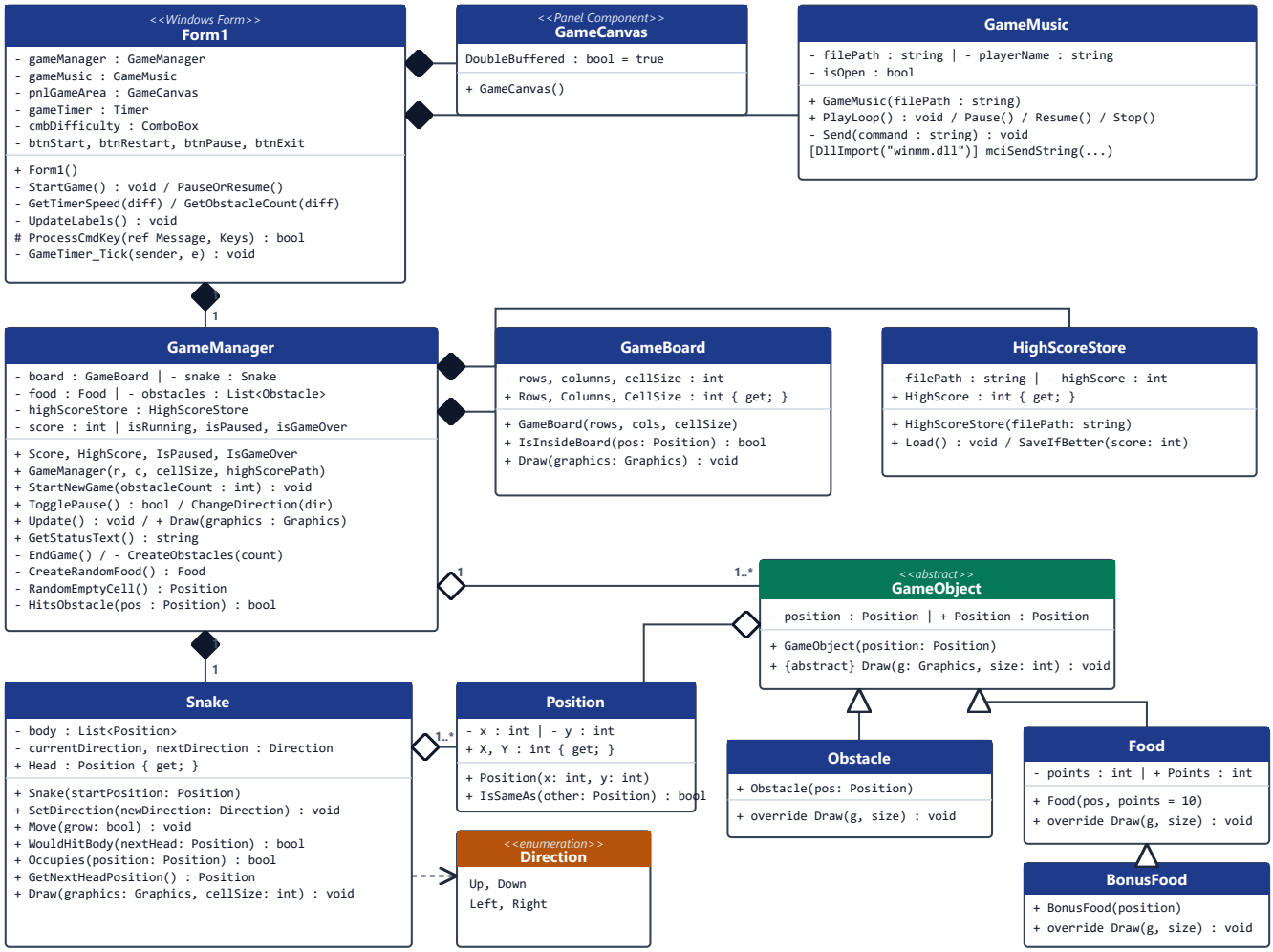


Figure 3.3: Paused State during Match (Timer halted, Music paused, Resume button active, Status: Paused)

4. UML Class Diagram

Structural UML diagram showing class hierarchy, visibilities (`+` public, `-` private, `#` protected), generalization, composition, and aggregation.



5. Explanation of OOP Concepts Used

The game design explicitly incorporates the four core pillars of Object-Oriented Programming (OOP) to achieve loose coupling, modularity, and high maintainability without procedural sprawl:

1. Encapsulation (Information Hiding & State Protection)

Internal variables are declared `private` across all classes to preserve object integrity and eliminate unauthorized state manipulation:

- In `Snake`, the body coordinates (`List<Position> body`) and directional state (`currentDirection`, `nextDirection`) are hidden. Movement is restricted to the `Move()` method.
- **Input Integrity:** `Snake.SetDirection()` strictly validates direction changes to prevent immediate 180-degree self-collisions (e.g., attempting to turn Down while moving Up is rejected).
- **Read-Only Properties:** Public properties with getters only (e.g., `GameManager.Score`, `GameBoard.Rows`, `Position.X`, `Food.Points`) expose state safely without allowing external alteration.

2. Inheritance (Hierarchical Code Reuse)

Inheritance structures domain objects into intuitive parent-child relationships, eliminating redundant declarations:

- `GameObject` serves as the abstract base class for all board items, providing shared spatial properties (`Position`) and requiring a standardized drawing contract.
- `Food : GameObject` inherits position logic and introduces food point value (default 10 points).
- `BonusFood : Food` extends `Food`, passing `base(position, 20)` to specialize score value and visual rendering while reusing all positioning mechanics.
- `Obstacle : GameObject` inherits spatial tracking while implementing distinct obstacle graphics.
- **Framework Inheritance:** `Form1` inherits from `Form` and `GameCanvas` inherits from `Panel`, enabling custom paint overrides and double-buffered window rendering.

3. Polymorphism (Dynamic Dispatch & Uniform Interface)

Polymorphism allows derived classes to override base contracts to deliver specialized behaviors through a common interface:

- **Method Overriding:** `GameObject` defines the abstract method `Draw(Graphics g, int size)`. Derived classes `Food` (red circle), `BonusFood` (orange circle), and `Obstacle` (gray square) override `Draw()` with customized rendering algorithms.
- **Uniform Execution:** `GameManager` handles food polymorphically: whether it holds an instance of `Food` or `BonusFood`, invoking `food.Draw(graphics, cellSize)` executes the appropriate specialized rendering routine without type casting.
- **System Event Interception:** `Form1` overrides `ProcessCmdKey` to intercept arrow key inputs before default Windows Form focus navigation can switch button states.

4. Abstraction (Separation of Concerns & Interface Simplicity)

Abstraction separates high-level domain operations from intricate low-level mechanics:

- `GameObject` provides an abstract representation of spatial grid entities without concerning itself with whether an object is edible, hazardous, or stationary.
- `GameManager` encapsulates game mechanics, collision math, and timer ticks. The presentation layer (`Form1`) interacts only through clean operations: `StartNewGame()`, `Update()`, and `ChangeDirection()`.
- `GameMusic` abstracts Win32 Multimedia API calls (`mciSendString` via P/Invoke) behind simple methods: `PlayLoop()`, `Pause()`, `Resume()`, and `Stop()`.

6. Instructions to Run the Project

Prerequisites: Visual Studio 2022 (v17.10+) or Visual Studio 2026 • .NET 10.0 SDK • Windows 10/11 OS (x64)

Method A: Running via Visual Studio

1. Open `SnakeGameFinal.slnx` in Visual Studio.
2. Set solution configuration to **Debug** and platform to **Any CPU**.
3. Press `F5` (or click **Start**) to build and run.

Method B: Running via Command Line (CLI)

1. Open terminal in the project directory:
   `cd c:\Users\vboxuser\Desktop\SnakeGameFinal`
2. Execute: `dotnet run`

Controls & Input Mapping

Input Key / Control	Function / In-Game Action
Up Arrow / **W**	Direct snake upward (reversal prevented if currently moving downward)
Down Arrow / **S**	Direct snake downward (reversal prevented if currently moving upward)
Left Arrow / **A**	Direct snake leftward (reversal prevented if currently moving rightward)
Right Arrow / **D**	Direct snake rightward (reversal prevented if currently moving leftward)
P Key	Pause / Resume the ongoing game session
Difficulty Dropdown	Select Easy (180ms, 0 obstacles), Medium (110ms, 4 obstacles), or Hard (70ms, 8 obstacles)
Start / Restart Buttons	Start a fresh match with randomized board layout and selected difficulty
Exit Button	Stop music playback and cleanly exit the application

7. Limitations & Future Improvements

Current Project Limitations

- **Platform Dependency:** Relies on Windows Forms and native Win32 MCI (`winmm.dll`), limiting portability to Microsoft Windows.
- **Fixed Grid Resolution:** Board size is fixed at 25×30 cells (20px each); responsive window resizing is disabled.
- **Local Persistence:** High score is saved in plain text (`highscore.txt`) without online leaderboard integration or encryption.

Proposed Future Improvements

- **Dynamic Power-Ups:** Introduce collectibles like speed boosts, slow-motion timers, and obstacle-destroyer pills.
- **Custom Themes & Audio:** Add selectable retro arcade, neon matrix, and dark-mode themes with customizable snake skins.
- **Multiplayer & AI Opponents:** Implement local split-screen 2-player mode and an autonomous AI snake driven by A* pathfinding.
- **Cross-Platform Port:** Migrate the UI layer to .NET MAUI or Avalonia UI to support macOS, Linux, and mobile platforms.